

Problem Set 0

Abdul Hadi Khan

Due January 18

1. [19] Consider this pseudocode for sorting a list of 0s and 1s so that all 0s are before all 1s.

```
 $n \leftarrow \text{length of } L$   
while true do  
  for  $i$  from 1 to  $n$  do  
    if  $L[i] = 1$  then  
      break  
  for  $j$  from  $n$  to 1 do  
    if  $L[j] = 0$  then  
      break  
  if  $j > i$  then  
    swap  $L[i], L[j]$   
  else  
    break
```

- (a) [2] Give a high-level description of the algorithm. It should be as short and clear as possible. You should avoid details that matter for runtime but not for correctness.

Solution:

The algorithm finds the first 1 from the left side of the list and the first 0 from the right side of the list. If the 1 comes before the 0 (meaning they're in the wrong order), the algorithm swaps them. It repeats this process until all the 0s are placed before all the 1s.

- (b) [17] Analyze the runtime of the pseudocode, when the input is a list of length $2n$ containing n 0s and n 1s. Fill out the table with O , Ω , and Θ bounds on the best-, worst-, and average-case runtime. The distribution for the average-case runtime is the uniform distribution on lists with n 0s and n 1s.

Give appropriate justifications. You don't need a formal proof for every entry. Some entries may not need any statement at all. The easy entries are worth 1 point, and the ones requiring more complex arguments are worth 3.

Solution:

	O	Ω	Θ
Best Case	$O(n)$	$\Omega(n)$	$\Theta(n)$
Worst Case	$O(n^2)$	$\Omega(n^2)$	$\Theta(n^2)$
Average Case	$O(n^2)$	$\Omega(n^2)$	$\Theta(n^2)$

Justifications:

Best Case: The best case happens when the list is already sorted, like $[0, 0, \dots, 0, 1, 1, \dots, 1]$ with all the 0s already before all the 1s. In this situation, the first loop looks through the list from the left and finds the first 1 somewhere around the middle (after going through about n elements). Then the second loop looks from the right and finds the last 0, which is also around the middle. Since the position of the last 0 is not greater than the position of the first 1, the algorithm realizes everything is already sorted and stops immediately. The algorithm only goes through the list once, looking at each element, so it takes $\Theta(n)$ time.

Worst Case: The worst case happens when the list is completely backwards, like $[1, 1, \dots, 1, 0, 0, \dots, 0]$ with all the 1s before all the 0s. This is bad because the algorithm needs to do a lot of work because each time it goes through the main loop, it finds one 1 from the left side and one 0 from the right side and swaps them. After the first swap, there's one 0 at the start and one 1 at the end. After the second swap, there are two 0s at the start and two 1s at the end, and so on. The algorithm needs to do exactly n swaps to fix everything. But the issue here is that for each swap, the algorithm has to scan through parts of the list to find where the next 1 and 0 are. Even though these scans get a bit shorter each time, they still take about $O(n)$ time on average for each. So we have n swaps, and each swap requires about n steps of work, giving us $n \times n = n^2$ total time. This means the worst case is $\Theta(n^2)$.

Average Case: For the average case we need to think about what happens when a random arrangement of n 0s and n 1s are mixed together. On average the algorithm still needs to do around n swaps to sort everything (the exact number depends on how mixed up things are, but it's in that ballpark). The key thing is that for each of these swaps, the algorithm still has to scan through the list to find the positions of the next 1 and 0 to swap. These scans take about n steps each time on average, so even though the number of swaps might vary depending on the specific random arrangement, we still end up doing roughly n swaps with each swap taking n time, giving us n^2 time overall. Therefore, the average case here is also $\Theta(n^2)$.

- (c) **[1 (Bonus)]** Give an implementation of the same algorithm with better worst-case and average-case runtime.

Solution:

We can make the original algorithm much faster using a new two pointer approach/algorithm. The idea is to have one pointer starting at the left side of the list and another pointer starting at the right side. The left pointer moves right looking for 1s, and the right pointer moves left looking for 0s. Whenever we find a 1 on the left side and a 0 on the right side, we swap them. We keep doing this until the two pointers meet in the middle. Here's the pseudocode for this:

```
left ← 1           ▷ Initialize left pointer (LP) at start of list
right ← 2n          ▷ Initialize right pointer (RP) at end of list
while left < right do   ▷ Continue until pointers meet or cross
    while left ≤ 2n and L[left] = 0 do   ▷ Move LP past all 0s
        left ← left + 1
    while right ≥ 1 and L[right] = 1 do   ▷ Move RP past all 1s
        right ← right - 1
    if left < right then   ▷ Swap elements if pointers not crossed
        swap L[left], L[right]
```

This approach/algorithm is much better because each pointer only moves through the list once. The left pointer starts at position 1 and can move at most to position $2n$. The right pointer starts at position $2n$ and can move at most to position 1. Since each pointer only visits each position once, the total work is just $\Theta(n)$ in all cases. It doesn't matter if the list is already sorted, completely backwards, or randomly mixed up, because we always just march through the list once with our two pointers, so it's always $\Theta(n)$ time. Thus this is much better than the original algorithm which took $\Theta(n^2)$ time in the worst and average cases.

2. [10] The following Python code gets a string containing a number, and returns a string with a list of all numbers up to the input number, separated by commas. What is the runtime? For this question, you are allowed (and encouraged!) to look up information online. Be careful, this one is tricky.

```
def list_of_numbers(num):
    n = int(num)
    out = ""
    for i in range(n):
        out += str(i+1) + ","
    return out
```

Solution:

Runtime: $\Theta(n^2 \log n)$

Explanation:

The problem here is with how Python handles strings because in Python

strings cannot be changed after they're created because they're **"immutable"**. So when we write `out += str(i+1) + ","`, Python can't just add the new characters to the end of the existing string, instead it has to create a brand new string and copy everything from the old string into it, and then add the new part.

As the loop runs, after the first iteration, `out` contains just `"1,"`. After the second iteration, Python creates a new string, copies `"1,"` into it and adds `"2,"`, making the `out` `"1,2,"`. After the third iteration, Python creates yet another new string, copies `"1,2,"` into it, and adds `"3,"`, so `out` becomes `"1,2,3,"`. The main thing to notice here is that every time we add something, we have to copy all the previous stuff.

By the time we're on iteration k , the string `out` already contains k numbers with commas. The string has gotten pretty long at this point since the length is now roughly $k \log k$ because larger numbers have more digits. For example, the number 100 has 3 digits, and $\log_{10} 100 = 2$, so the number of digits grows logarithmically with the value. Thus, when we do the concatenation on iteration k , Python has to copy this entire string of length $k \log k$ which takes $k \log k$ time.

Hence on the first iteration we copy a string of length about $1 \log 1$. On the second iteration we copy about $2 \log 2$. On the third, about $3 \log 3$, and so on. In this way, by the time we get to iteration n , we're copying about $n \log n$ characters. Adding all these up, we get: $1 \log 1 + 2 \log 2 + 3 \log 3 + \dots + n \log n$. We can then pull out the $\log n$ (since $\log k$ is roughly $\log n$ for large k), and we're left with $\log n \times (1 + 2 + 3 + \dots + n)$. From here, the sum $1 + 2 + 3 + \dots + n$ equals $\frac{n(n+1)}{2}$ because of the summation formula, which is about $\frac{n^2}{2}$. Thus the total time is roughly $\log n \times \frac{n^2}{2} = \frac{n^2 \log n}{2}$, which we write as $\Theta(n^2 \log n)$ because constant factors like $\frac{1}{2}$ don't have any affect on the asymptotic growth rate in Big-Theta notation.